Everything Comes Back (Eventually)

Poincaré recurrence in a box of gas

PH 217 Course Project

[Name 1] ([Roll no.]) [Name 2] ([Roll no.]) [Name 3] ([Roll no.])

Department of Physics, IIT Bombay

November 2026

Abstract

Poincaré's recurrence theorem says that a system with finite room in phase space, whose motion never squeezes phase-space volume, returns again and again as close as we like to where it started. We prove the theorem and show why every Hamiltonian system trapped in a bounded region obeys it. We then test it on three systems. Two short warm-ups (a pair of oscillators and Arnold's cat map) show that recurrence survives both non-periodic and chaotic motion. Our main case is a gas of N free particles in a box, all starting in the left half. We map the gas onto a single point moving in a straight line on an N-dimensional torus. On this torus the event "all particles on the left" takes up a fraction 2^{-N} of the space, so Kac's lemma predicts a mean waiting time of 2^N looks. Our simulations for $N = 1$ to 14 agree with this to within two standard errors at every N. For $N = 100$ the same formula gives a wait of about three billion times the age of the universe. That is how recurrence and the second law of thermodynamics manage to live together.

Contents

1 Motivation **2**

2 The recurrence theorem **2**

3 Why Hamiltonian systems qualify **3**

4 Two warm-up examples **5**

5 Main case: a gas in a box **6**

6 Interpretation **11**

7 Conclusion **13**

A Reproducing the results **13**

B Code **14**

1 Motivation

Suppose we open a box and all the air rushes into one corner. Nobody has ever seen this happen, and the second law of thermodynamics says it should not. Yet in 1890 Poincaré proved a theorem that seems to say it must happen, sooner or later [1]. In 1896 Zermelo used exactly this theorem to attack Boltzmann's kinetic theory of gases [2]. If every state comes back, he argued, entropy cannot keep going up. Boltzmann's reply was about time [3]. The state does come back, but for a real gas the wait is so long that it never matters.

This project puts numbers on that argument. We ask two questions.

1. **Why** must a closed mechanical system come back near its starting state?
2. **How long** do we have to wait?

The first question is answered by Poincaré's theorem and Liouville's theorem (Sections 2 and 3). The second is answered by Kac's lemma (Section 5.7). We check both answers with our own simulations. The main test is a gas in a box, chosen because it is simple enough to solve exactly and still shows the full story, from a quick return for one particle to a wait longer than the universe for a hundred.

2 The recurrence theorem

Definition 1. Let Ω be a space with a volume μ (a measure). A map $T : \Omega \to \Omega$ is *measure preserving* if $\mu(T^{-1}B) = \mu(B)$ for every measurable set $B \subset \Omega$.

For an invertible map this just says that T moves sets around without stretching or squeezing their volume.

Theorem 1 (Poincaré recurrence)**.** *Let T be measure preserving on Ω with $\mu(\Omega) < \infty$. Then for every set A with $\mu(A) > 0$, almost every point $x \in A$ returns to A infinitely often. That is, $T^n x \in A$ for infinitely many $n \geq 1$.*

"Almost every" means that the points which fail form a set of zero volume. We could never land on one by picking a starting point at random. For a flow ϕ_t in continuous time we apply the theorem to the map $T = \phi_\tau$ for any fixed step $\tau > 0$.

2.1 Proof

Step 1: almost every point returns at least once. Let

$$B = \{\, x \in A \;:\; T^n x \notin A \text{ for all } n \geq 1 \,\}$$

be the points of A that never come back. We claim the sets B, $T^{-1}B$, $T^{-2}B$, $\ldots$ never overlap. Suppose x lies in both $T^{-m}B$ and $T^{-n}B$ with $m < n$. Put $y = T^m x$. Then $y \in B$, and $T^{n-m}y = T^n x \in B \subset A$. So y comes back to A after $n - m$ steps, which is impossible for a point of B.

Each of these sets has volume $\mu(T^{-k}B) = \mu(B)$ because T preserves volume. They are disjoint and all sit inside Ω, so for every K

$$(K+1)\,\mu(B) \;=\; \sum_{k=0}^{K} \mu\Big(T^{-k}B\Big) \;\leq\; \mu(\Omega) \;<\; \infty.$$

The left side grows without limit unless $\mu(B) = 0$. So $\mu(B) = 0$.

Step 2: almost every point returns infinitely often. A point of A that returns only finitely often has a last visit to A, at some step $m \geq 0$. Then $T^m x \in B$, so $x \in T^{-m}B$. All such points therefore lie in $\bigcup_{m \geq 0} T^{-m}B$. This is a countable union of sets of volume zero, so it has volume zero too. □

The proof is a pigeonhole argument. Infinitely many copies of a set with positive volume cannot fit side by side in a box of finite volume, so two of them must overlap. Figure 1 shows this with a real area-preserving map.

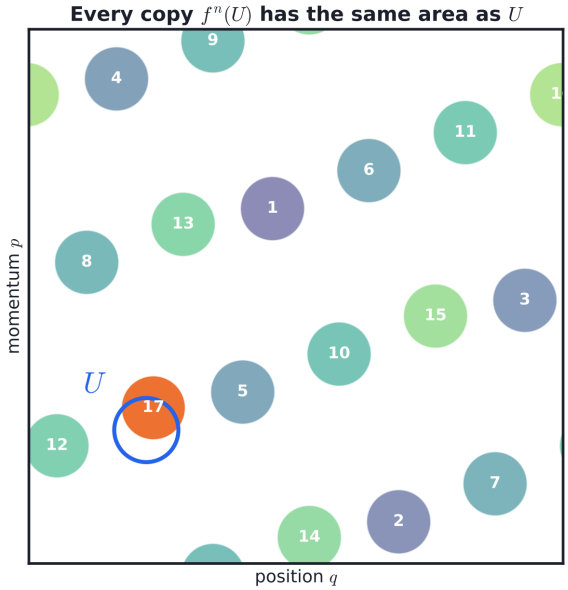


Figure 1: The pigeonhole idea. A disk U is moved by the area-preserving shift $(q, p) \mapsto (q + \sqrt{5} - 2,\ p + \sqrt{2} - 1) \bmod 1$. Each copy $f^n(U)$ has the same area as U (here about 1.1% of the box). At most about 88 such copies could fit without overlapping. In practice copy 17 already lands on U.

3 Why Hamiltonian systems qualify

For the theorem to apply to physics, we need a mechanical system with both properties: no squeezing and finite room.

3.1 No squeezing: Liouville's theorem

Take a system with coordinates q_i and momenta p_i, $i = 1, \ldots, n$, obeying Hamilton's equations

$$\dot{q}_i = \frac{\partial H}{\partial p_i}, \qquad \dot{p}_i = -\frac{\partial H}{\partial q_i}.$$

These define a velocity field in the $2n$-dimensional phase space. Its divergence is

$$\nabla \cdot (\dot{q}, \dot{p}) \;=\; \sum_{i=1}^{n} \left(\frac{\partial \dot{q}_i}{\partial q_i} + \frac{\partial \dot{p}_i}{\partial p_i} \right) \;=\; \sum_{i=1}^{n} \left(\frac{\partial^2 H}{\partial q_i \, \partial p_i} - \frac{\partial^2 H}{\partial p_i \, \partial q_i} \right) \;=\; 0,$$

since mixed partial derivatives are equal. A region $R(t)$ carried along by the flow changes volume at the rate

$$\frac{dV}{dt} = \int_{R(t)} \nabla \cdot (\dot{q}, \dot{p}) \; d^n q \, d^n p = 0.$$

So every volume in phase space is kept fixed. This is Liouville's theorem [5], and it is exactly the "measure preserving" condition.

3.2 Finite room: bounded energy

Energy is conserved, so the motion stays on the surface $H = E$. If this surface is bounded (for example, a gas held in a closed box at fixed energy, where walls bound the positions and the energy bounds the momenta), the motion lives in a region of finite volume. Then Poincaré's theorem applies.

3.3 What friction does

Add a friction force, $\dot{p} = -\partial H/\partial q - \gamma p$. The divergence becomes $-\gamma$, so $dV/dt = -\gamma V$ and

$$V(t) = V(0)\, e^{-\gamma t}.$$

Every region shrinks onto the resting state and nothing comes back. Friction breaks the theorem.

3.4 Numerical check

We tested this on a pendulum, $H = p^2/2 - \cos q$. We filled a disk of radius 0.45 centred at $(q, p) = (0, 1.55)$ with starting points and moved each one with `scipy`'s DOP853 integrator [9] (relative tolerance 10^{-10}). The area was measured from 20,000 points on the boundary with the shoelace formula. Table 1 and Figure 2 show the result.

case	area at $t = 0$	area at $t = 12$	theory at $t = 12$
no friction	0.63617	0.63616	0.63617
friction, $\gamma = 0.25$	0.63617	0.03167	0.03167

Table 1: Area of a blob of pendulum states. The tiny drift without friction comes from the finite number of boundary points once the blob becomes a very thin swirl.

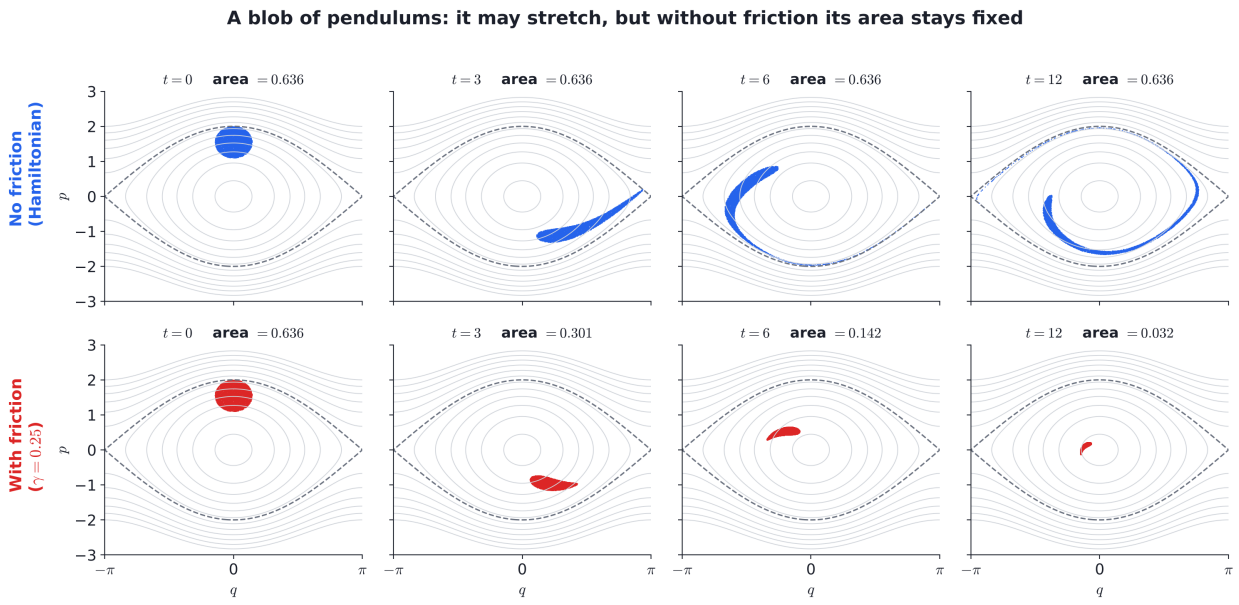


Figure 2: A blob of pendulum states. Top row: without friction it swirls into a long thin shape but keeps its area. Bottom row: with friction it shrinks to the bottom of the well. Dashed line: the separatrix.

4 Two warm-up examples

Before the gas, two examples show what recurrence is and what it is not.

4.1 Two oscillators with an irrational frequency ratio

Take two uncoupled oscillators, $H = \frac{1}{2}(p_1^2 + \omega_1^2 x_1^2) + \frac{1}{2}(p_2^2 + \omega_2^2 x_2^2)$. Each keeps its own energy, so the state is fixed by two phases $\theta_i = \omega_i t \bmod 2\pi$. The motion is a straight line on a 2-torus, the surface of a doughnut. We chose $\omega_2/\omega_1 = \varphi = (1+\sqrt{5})/2$, the golden ratio. Since this is irrational, the line never closes. The motion is never periodic. Even so, by Weyl's theorem [8] it passes arbitrarily close to every point of the torus, including its starting point. This is recurrence without repetition (Figure 3).

How long until it comes back within ε? On an n-torus a straight-line flow with velocity $\boldsymbol{\omega}$ spreads evenly, so the fraction of time spent inside a ball B_ε of volume V_ε is $V_\varepsilon/(2\pi)^n$. A single pass through the ball takes, on average, the mean chord length divided by the speed. For straight lines crossing a convex body, the mean chord is $V_\varepsilon/\sigma_\varepsilon$, where σ_ε is the cross-section the flow sees (2ε in two dimensions, $\pi\varepsilon^2$ in three). The mean time between entries is the time per visit divided by the fraction of time inside:

$$T(\varepsilon) = \frac{V_\varepsilon/(\sigma_\varepsilon|\boldsymbol{\omega}|)}{V_\varepsilon/(2\pi)^n} = \frac{(2\pi)^n}{|\boldsymbol{\omega}|\,\sigma_\varepsilon} \;\propto\; \varepsilon^{-(n-1)}.$$

Every extra oscillator multiplies the wait by another factor of $1/\varepsilon$. Our simulations match this formula to within 1% for two oscillators and 3% for three (Figure 3, right). Keep this in mind for the gas: more parts means much longer waits.

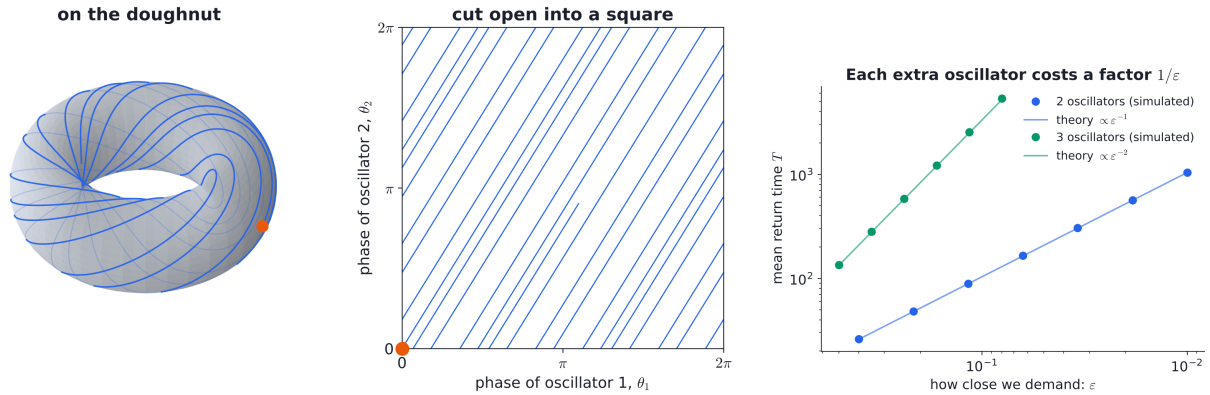


Figure 3: Left: two oscillators with frequency ratio φ wind round a torus without ever closing. Right: mean return time into an ε-ball for two and three oscillators (points) against the formula above (lines).

4.2 Arnold's cat map: chaos does not stop recurrence

The cat map [6] acts on the unit square with opposite edges glued:

$$\begin{pmatrix} x \\ y \end{pmatrix} \mapsto \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \bmod 1.$$

The matrix has determinant 1, so area is preserved. Its eigenvalues are $\lambda_\pm = (3 \pm \sqrt{5})/2 \approx 2.618$ and 0.382. Lengths along one direction grow by a factor of 2.618 every step, so nearby points separate exponentially fast, with Lyapunov exponent $\ln 2.618 \approx 0.96$. The map is chaotic.

We applied it to a picture on an $M \times M$ grid of pixels, using the same rule mod M. On the grid the map is a one-to-one shuffle of M^2 pixels. Any shuffle of a finite set, repeated enough times,

must return to where it began. For $M = 101$ the picture returns exactly after 25 steps (Figure 4). Dyson and Falk showed that the period is never more than $3M$ [7]. Chaos changes how fast things get mixed up. It does not stop them coming back. Cats always come home, this one just takes the scenic route.

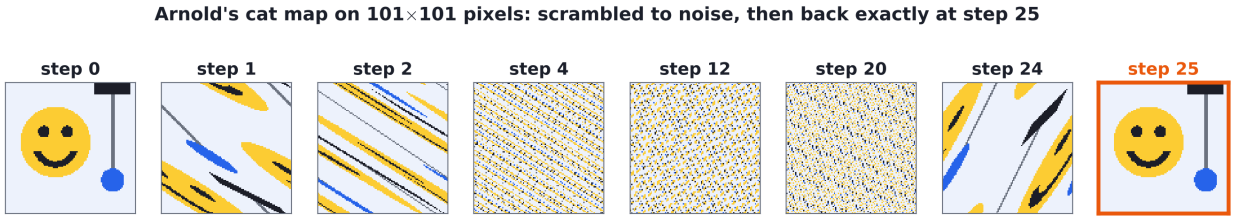


Figure 4: Arnold's cat map on a 101×101 grid. By step 4 the picture looks like noise. At step 25 every pixel is back in place.

5 Main case: a gas in a box

5.1 Setup

We place N particles in a square box of side 1. They do not interact with each other and they bounce off the walls elastically. All start in the left half, $x < 1/2$, with random positions and velocities drawn from a normal distribution. We look at the box once per unit of time ($\Delta t = 1$) and ask: at which looks are *all* the particles back in the left half? Since this question only involves the x coordinates, from now on we only track x.

5.2 Each bounce is a circle

A particle bouncing between $x = 0$ and $x = 1$ with speed v can be "unfolded". Let $\theta(t) = (x_0 + vt) \bmod 2$ and

$$x = \begin{cases} \theta, & 0 \le \theta < 1, \\ 2 - \theta, & 1 \le \theta < 2. \end{cases}$$

While $\theta < 1$ the particle moves with velocity $+v$, and while $\theta > 1$ it moves with $-v$. The switch at $\theta = 1$ is the bounce off the right wall, and the wrap from $\theta = 2$ back to 0 is the bounce off the left wall. So the bouncing particle is the same as a point running round a circle of length 2. Half a lap takes it from one wall to the other.

To make the left half of the box a single piece of the circle, we shift the angle by a quarter turn:

$$\varphi = \left(\theta + \tfrac{1}{2}\right) \bmod 2.$$

The particle is on the left ($x < 1/2$) when $\theta \in [0, \frac{1}{2}) \cup (\frac{3}{2}, 2)$. Adding $\frac{1}{2}$ sends these to $[\frac{1}{2}, 1)$ and $(0, \frac{1}{2})$, so

$$x < \tfrac{1}{2} \quad \Longleftrightarrow \quad \varphi \in [0, 1),$$

up to the single point $\varphi = 0$, which has zero length. Figure 5 (left) shows the circle with its left half in blue.

5.3 The gas is a point on a torus

With N particles we have N angles. The state of the gas is the point

$$\boldsymbol{\varphi} = (\varphi_1, \ldots, \varphi_N) \in \mathbb{T}^N = [0, 2)^N,$$

where opposite faces of the cube $[0, 2)^N$ are glued together. This is the N-dimensional torus. Between two looks every angle moves forward by its own velocity, so one look is the map

$$T(\boldsymbol{\varphi}) = (\boldsymbol{\varphi} + \mathbf{v}) \bmod 2, \qquad \mathbf{v} = (v_1, \ldots, v_N).$$

The whole gas is a single point jumping in a straight line across the torus. For $N = 2$ the torus is a 2×2 square with opposite edges glued, which is a doughnut (Figure 5). For $N = 3$ it is a cube with opposite faces glued. We will not try to draw the ten-dimensional one.

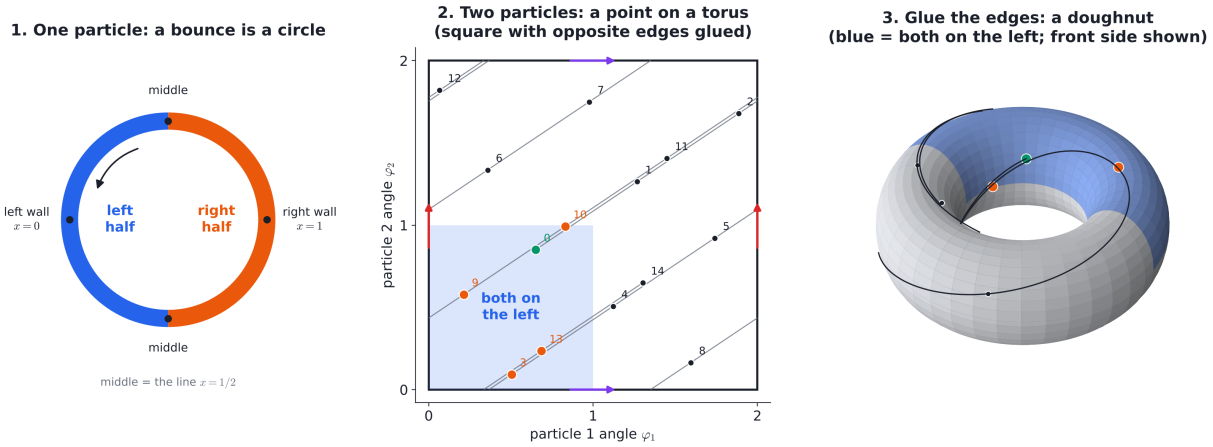


Figure 5: From the box to the torus, for $N = 2$. Left: one particle's bounce is a circle, with the left half of the box in blue. Middle: two particles are a point in a square with opposite edges glued. Numbered dots are the looks. Orange dots are looks where both particles are on the left. The first return is at look 3. Right: gluing the edges gives a doughnut, with the target region in blue.

5.4 Poincaré's two conditions

No squeezing. T is a translation. It moves every set without changing its shape, so $\mu(T^{-1}B) = \mu(B)$ for the usual volume μ.

Finite room. The torus has volume $\mu(\mathbb{T}^N) = 2^N < \infty$.

Both conditions hold, so the gas must keep coming back to "all on the left". This is the same argument as Section 3, but here we can see it directly. Free motion in a box is Hamiltonian, and the translation on the torus is its Liouville flow written in convenient coordinates.

5.5 Ergodicity: the point fills the torus evenly

For Kac's lemma we also need the motion to be *ergodic*. Roughly, this means a typical point wanders through the whole space instead of staying in one part. For a translation on a torus there is a clean test due to Weyl [8]. The map $\boldsymbol{\varphi} \mapsto \boldsymbol{\varphi} + \mathbf{v} \bmod 2$ is ergodic when the numbers $v_1/2, \ldots, v_N/2$ and 1 are rationally independent. In other words, no relation $k_1 v_1/2 + \cdots + k_N v_N/2 = m$ holds with whole numbers k_i, m unless all of them are zero. Velocities drawn from a continuous distribution meet this condition with probability 1.

When the map is ergodic, the looks spread evenly over the torus. The share of looks that land in a region equals that region's share of the volume. Figure 6 shows this for $N = 2$. After 400 looks, 97 land in the quarter of the square where both particles are on the left. That is one look in $400/97 \approx 4.1$.

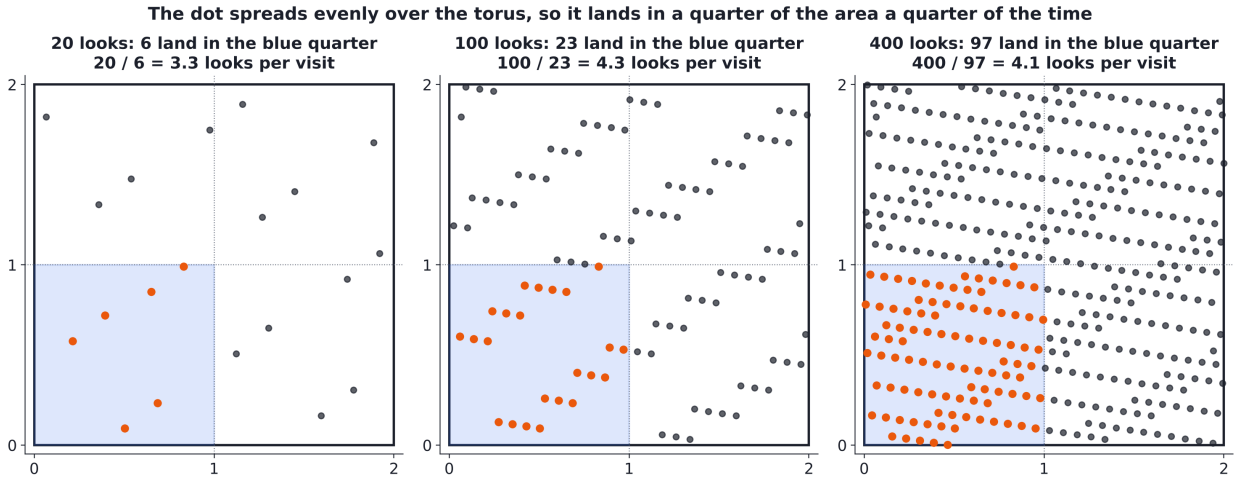


Figure 6: The looks of a two-particle gas fill the square torus evenly. The blue quarter is "both particles on the left". As the number of looks grows, the share of looks inside it settles at 1/4.

5.6 The size of the target: 2^{-N}

"All particles on the left" means $\varphi_i \in [0, 1)$ for every i, so the target is the cube

$$A = [0, 1)^N \subset \mathbb{T}^N.$$

Volume is a product measure. For a set built by choosing one interval for each coordinate, the volume is the product of the lengths. So

$$\mu(A) = \underbrace{1 \times 1 \times \cdots \times 1}_{N} = 1, \qquad \frac{\mu(A)}{\mu(\mathbb{T}^N)} = \frac{1}{2^N} = 2^{-N}.$$

This product rule works because the target is a "box" set. Each particle's condition only involves its own angle, which is the same as saying the particles move independently. It would not work for a mixed condition such as "exactly half of the particles on the left".

Geometrically, each new particle adds one more direction to the torus, and the target keeps only the half of that direction where the new particle is on the left. So the target halves every time (Figure 7).

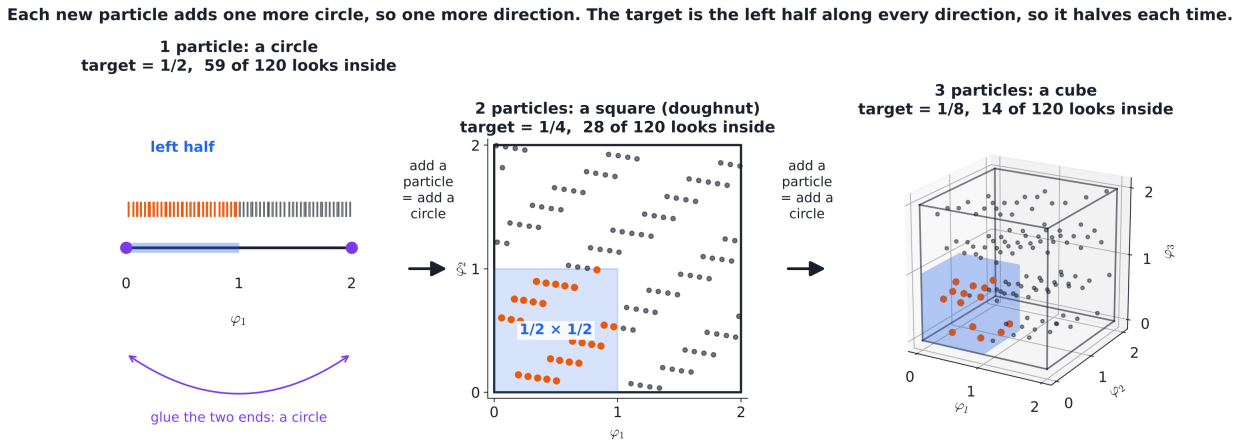


Figure 7: Going up one dimension at a time with the same 120 looks. One particle: a circle, target 1/2, 59 looks inside. Two particles: a square, target 1/4, 28 inside. Three particles: a cube, target 1/8, 14 inside.

5.7 How long to wait: Kac's lemma

Poincaré tells us the gas comes back. Kac's lemma tells us how often [4].

Lemma 2 (Kac, 1947)**.** *Let T be measure preserving and ergodic on Ω with $\mu(\Omega) < \infty$, and let $\mu(A) > 0$. Then the mean return time to A, averaged over starting points in A, is*

$$\langle T_A \rangle = \frac{\mu(\Omega)}{\mu(A)}.$$

Proof sketch. Normalise so that $\mu(\Omega) = 1$. Split A into pieces A_n, where A_n holds the points whose first return to A happens at step n. A point of A_n visits the sets $A_n, TA_n, \ldots, T^{n-1}A_n$ before it gets back. These sets, over all n and all $0 \le k < n$, do not overlap, and by ergodicity they fill Ω up to zero volume. Each $T^k A_n$ has volume $\mu(A_n)$, so

$$\sum_{n \ge 1} n\, \mu(A_n) = \mu(\Omega) = 1.$$

The mean return time is $\sum_n n\, \mu(A_n)/\mu(A) = 1/\mu(A)$. □

There is also a quick counting argument. In a long run of L looks, the point lands in A about $\mu(A)L/\mu(\Omega)$ times. The gaps between these visits fill the run, so the mean gap is about L divided by the number of visits, which is $\mu(\Omega)/\mu(A)$.

For the gas, $\mu(\mathbb{T}^N)/\mu(A) = 2^N$, so

$$\boxed{\langle T \rangle = 2^N \text{ looks}}$$

One particle comes back to the left every 2 looks on average, two particles every 4, three every 8, and each extra particle doubles the wait (Figure 8).

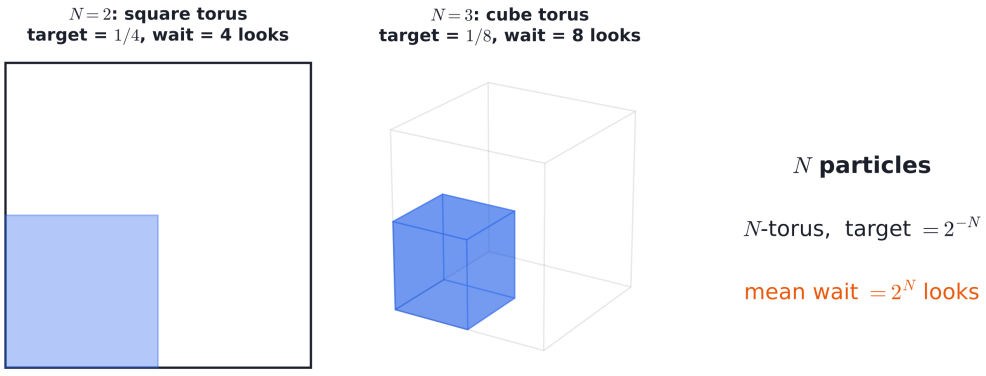


Figure 8: The target is half of each direction, so it is $1/4$ of the square torus, $1/8$ of the cube torus, and 2^{-N} of the N-torus. The mean wait is the inverse.

5.8 Numerical method

Because the particles are free, we never need to integrate equations of motion. The position at look k comes straight from the unfolding of Section 5.2, $x_i(k) = \text{bounce}(x_i(0) + v_i k)$. So there is no build-up of integration error, however long the run. For each N from 1 to 14 we:

1. drew starting positions uniformly in $[0, \frac{1}{2})$ and velocities from a standard normal distribution (random seed 7),
2. computed all positions at looks $k = 1, 2, \ldots, K$ with $K = \min(4000 \cdot 2^N,\ 6 \times 10^7)$, in chunks of 10^6 looks to save memory,

3. recorded every look at which all $x_i < 1/2$, and took the gaps between consecutive such looks,
4. reported the mean gap and its standard error $s/\sqrt{n}$, where s is the sample standard deviation of the gaps and n the number of gaps.

This gives about 4000 returns for every N (3557 for $N = 14$, where the run length hit the cap). The standard error treats the gaps as independent, which is a slight simplification since consecutive gaps of a translation are correlated. The full code is in Appendix B.

5.9 Results

Figure 9 shows ten particles spreading out and then, at look 89, all being on the left again. That single wait is much shorter than the Kac average of $2^{10} = 1024$ looks. Individual waits vary a lot. Only the average is fixed.

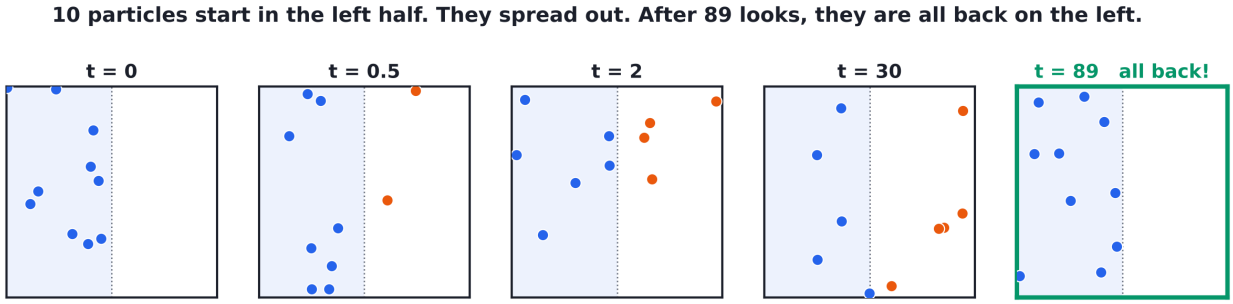


Figure 9: Ten particles start in the left half (blue). They spread out. At look 89 they are all on the left again.

Figure 10 follows the fraction of particles on the left over 6000 looks. With $N = 10$ it reaches 1 five times. With $N = 40$ it never does, which is no surprise, since the expected wait is about $2^{40} \approx 10^{12}$ looks.

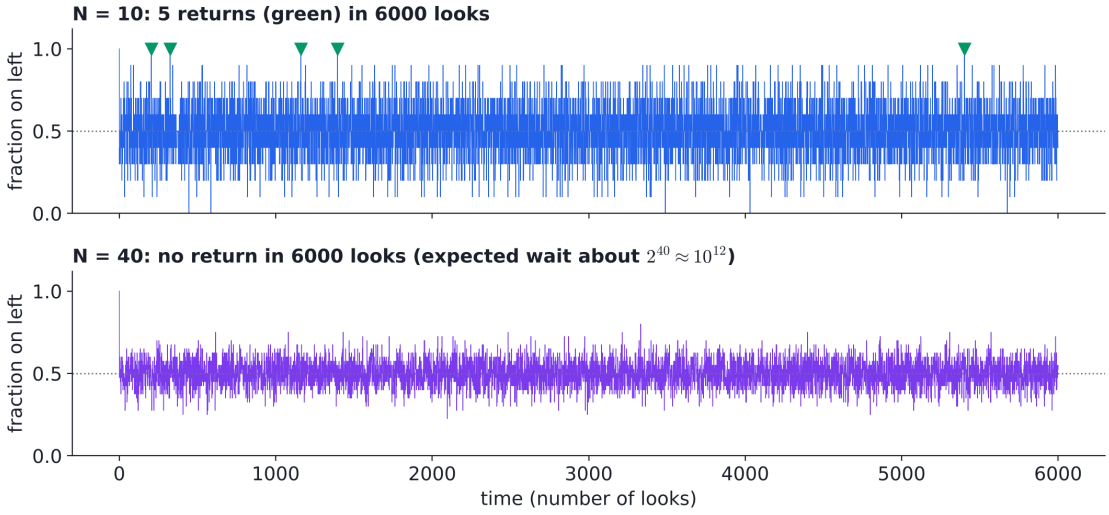


Figure 10: Fraction of particles in the left half over 6000 looks. Green markers: all particles on the left. The band for $N = 40$ is also much narrower than for $N = 10$ (Section 6.2).

Table 2 and Figure 11 compare the measured mean wait with Kac's prediction for $N = 1$ to 14. Over this range the wait grows by a factor of about 8000. The measured values agree with 2^N to within two standard errors at every N. The largest deviation is 1.77 standard errors, at $N = 5$.

With 14 points we expect about two to sit more than 1.5 standard errors away by chance. We see four. That is a little high, and it fits the slight underestimate of the error bars noted above.

N	returns counted	measured $\langle T \rangle$	Kac 2^N	deviation (std. errors)
1	4000	2.00 ± 0.01	2	−0.04
2	4000	4.00 ± 0.05	4	−0.01
3	4014	7.97 ± 0.25	8	−0.11
4	3998	16.00 ± 0.17	16	+0.02
5	3917	32.7 ± 0.4	32	+1.77
6	4009	63.8 ± 0.9	64	−0.21
7	4074	125.6 ± 1.5	128	−1.53
8	3956	258.7 ± 4.0	256	+0.69
9	3984	513.7 ± 7.1	512	+0.24
10	3904	1049 ± 16	1024	+1.57
11	4031	2031 ± 31	2048	−0.54
12	4023	4070 ± 76	4096	−0.34
13	4020	8148 ± 122	8192	−0.36
14	3557	16866 ± 292	16384	+1.65

Table 2: Mean number of looks between returns to “all particles on the left”. Raw numbers are in `figures/05_results.csv`.

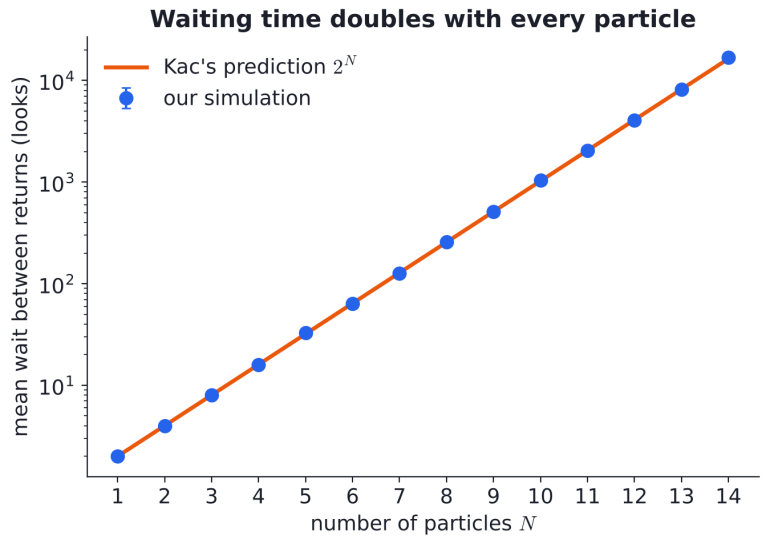


Figure 11: Measured mean wait (points with error bars) against Kac’s prediction 2^N (line), on a log scale. The wait doubles with every particle.

6 Interpretation

6.1 How long is “eventually”?

Kac’s formula lets us go far beyond what we can simulate. Table 3 assumes one look every millisecond.

The age of the universe is about 1.4×10^{10} years, so a hundred particles already need about three billion universe lifetimes. The air in a room holds roughly 10^{27} molecules. Its wait, $2^{10^{27}}$ looks, is a number with about 3×10^{26} digits. Nobody in the room needs to worry about breathing.

N	mean wait 2^N looks	at one look per millisecond
10	1.0×10^3	about 1 second
20	1.0×10^6	about 17 minutes
30	1.1×10^9	about 12 days
40	1.1×10^{12}	about 35 years
60	1.2×10^{18}	about 37 million years
100	1.3×10^{30}	about 4×10^{19} years

Table 3: Mean wait for all N particles to be in the left half at once.

6.2 Why a big gas looks evenly spread

Recurrence is not only rare for a large gas. The gas also rarely strays far from half and half. Because the looks fill the torus evenly and the volume is a product measure, each particle is on the left in an independent half of the looks. The number on the left is then binomial with mean $N/2$, and the fraction on the left has mean $1/2$ and standard deviation

$$\sigma = \frac{1}{2\sqrt{N}}.$$

From the runs in Figure 10 we measured $\sigma = 0.158$ for $N = 10$ and 0.079 for $N = 40$, matching the formula (0.158 and 0.079). For $N = 10^{27}$ the spread is about 2×10^{-14}. A real gas always looks evenly spread, even though it is guaranteed to come back to every unusual state given enough time.

6.3 Recurrence and the second law

This resolves the Zermelo–Boltzmann argument. Poincaré's theorem is correct, and so is the observed rise of entropy. The two apply on very different time scales. Over any time we could ever wait, an isolated large gas moves towards the most likely, evenly spread states and stays there. A return to a special state such as "all on the left" is certain in principle but takes far longer than the age of the universe. The second law is a statement about what is overwhelmingly likely, not about what is impossible.

6.4 Limits of our model

- **No collisions.** Our particles pass through each other. A real gas with collisions is still Hamiltonian, and at fixed energy in a closed box its energy surface still has finite volume. So Poincaré's theorem still holds. But the motion is no longer a translation on a torus, the target is no longer an exact product set, and we would have to measure its volume on the energy surface instead.
- **Only x matters.** We ignored the y motion because the target only depends on x. A target involving both coordinates would need a $2N$-dimensional torus.
- **Ergodicity assumed.** With exactly rational velocity ratios the gas would move on a lower-dimensional piece of the torus and the count would change. Our velocities are floating point numbers, which are technically rational, but their "period" is astronomically long compared with our runs, so the even filling holds over the run lengths we use.
- **Mean, not timetable.** Kac's lemma fixes the average wait only. Single waits vary a lot (look 89 against an average of 1024 for $N = 10$).

7 Conclusion

We proved Poincaré's recurrence theorem with a pigeonhole argument and showed, through Liouville's theorem, that every Hamiltonian system confined to a bounded region satisfies it. Two warm-up examples showed that recurrence does not need periodic motion and survives chaos. For a gas of N free particles in a box, unfolding each bounce turns the gas into one point moving in a straight line on an N-torus. The event "all particles on the left" is a cube of relative volume 2^{-N} on this torus, so Kac's lemma predicts a mean wait of 2^N looks. Our simulations confirm this for $N = 1$ to 14, and they also confirm the $1/(2\sqrt{N})$ size of the fluctuations. Recurrence is guaranteed. For anything of everyday size it is just slower than slow.

References

[1] H. Poincaré, "Sur le problème des trois corps et les équations de la dynamique", *Acta Mathematica* **13**, 1–270 (1890).

[2] E. Zermelo, "Ueber einen Satz der Dynamik und die mechanische Wärmetheorie", *Annalen der Physik* **57**, 485–494 (1896).

[3] L. Boltzmann, "Entgegnung auf die wärmetheoretischen Betrachtungen des Hrn. E. Zermelo", *Annalen der Physik* **57**, 773–784 (1896).

[4] M. Kac, "On the notion of recurrence in discrete stochastic processes", *Bulletin of the American Mathematical Society* **53**, 1002–1010 (1947).

[5] H. Goldstein, C. Poole and J. Safko, *Classical Mechanics*, 3rd ed., Addison-Wesley (2002), Section 9.9.

[6] V. I. Arnold and A. Avez, *Ergodic Problems of Classical Mechanics*, Benjamin (1968).

[7] F. J. Dyson and H. Falk, "Period of a discrete cat mapping", *American Mathematical Monthly* **99**, 603–614 (1992).

[8] H. Weyl, "Über die Gleichverteilung von Zahlen mod. Eins", *Mathematische Annalen* **77**, 313–352 (1916).

[9] E. Hairer, S. P. Nørsett and G. Wanner, *Solving Ordinary Differential Equations I*, 2nd ed., Springer (1993).

A Reproducing the results

All code, figures and raw numbers are in the repository `https://github.com/shishirvsh285/Poincare`. With Python 3 and the packages in `requirements.txt` (numpy, scipy, matplotlib, pillow):

```
pip install -r requirements.txt
python src/make_all.py          # rebuilds every figure, a few minutes
python src/05_gas_in_box.py     # or one script at a time
```

A Colab notebook that runs everything in the browser is in `notebook/`.

script	what it produces
`01_proof_pigeonhole.py`	Figure 1
`02_liouville_blob.py`	Table 1, Figure 2
`03_torus_oscillators.py`	Figure 3
`04_cat_map.py`	Figure 4
`05_gas_in_box.py`	Table 2, Figures 9, 10, 11
`06_gas_torus.py`	Figures 5, 8
`07_torus_figures.py`	Figures 6, 7
`style.py`	shared colours and fonts for all plots

B Code

We list the code for the main case in full. The code for the warm-up examples is in the repository.

B.1 `05_gas_in_box.py`: simulation and Kac test

```
"""
MAIN NUMERICAL CASE: a gas in a box, and how long it takes to "un-mix".

Setup
  N non-interacting particles in a square box of side 1, bouncing off the
  walls (elastic). All start in the LEFT half. We watch them spread out
  and ask: when are they all back in the left half?

Why this is a Poincare system
  Each particle's x-motion is a bounce between walls. Unfold the bounce and
  it becomes a point running round a circle of length 2:
      theta(t) = theta0 + v t  (mod 2),     x = theta if theta < 1 else 2 - theta
  So the full x-state is a point on an N-dimensional torus, moving in a
  straight line. That flow preserves volume and the torus is finite, so
  Poincare says the gas MUST come back to the left half, again and again.

What we measure (the prediction we test)
  Look at the box once per time step. Let A = "all N particles in the left
  half". A takes up a fraction mu(A) = (1/2)^N of the torus. Kac's lemma
  (the quantitative version of Poincare) says the AVERAGE number of steps
  between visits to A is exactly

      <T> = 1 / mu(A) = 2^N.

  We simulate long runs, measure <T> for N = 1..14, and compare.

Outputs
  figures/05_gas_snapshots.png   the box at a few moments, ending at a return
  figures/05_left_fraction.png   fraction of particles on the left vs time, N=10 vs N=40
  figures/05_kac_scaling.png     measured mean return time vs 2^N
  figures/05_gas.gif             animation of the spread and the return
  figures/05_results.csv         the numbers behind the Kac plot
"""
import csv

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter
from matplotlib.patches import Rectangle

import style

DT = 1.0                # we look at the box once per unit of time
RNG_SEED = 7


# ---------------------------------------------------------------- physics
def bounce(theta):
    """Unfolded coordinate on a circle of length 2 -> position in the box [0, 1]."""
    th = theta % 2.0
```

```python
    return np.where(th < 1.0, th, 2.0 - th)


def make_gas(n, rng):
    """Random start with everyone in the left half, Maxwell-like random velocities."""
    x0 = rng.uniform(0, 0.5, n)
    y0 = rng.uniform(0, 1.0, n)
    vx = rng.normal(0, 1, n)
    vy = rng.normal(0, 1, n)
    return x0, y0, vx, vy


def positions(x0, y0, vx, vy, t):
    t = np.atleast_1d(t)[:, None]
    return bounce(x0 + vx * t), bounce(y0 + vy * t)


def visits_to_left_half(x0, vx, n_steps, chunk=1_000_000):
    """Step indices k (k >= 1) at which ALL particles have x < 1/2."""
    hits = []
    for start in range(1, n_steps + 1, chunk):
        k = np.arange(start, min(start + chunk, n_steps + 1))
        x = bounce(x0[None, :] + vx[None, :] * (k[:, None] * DT))
        hits.append(k[np.all(x < 0.5, axis=1)])
    return np.concatenate(hits)


# ---------------------------------------------------------------- figures
def draw_box(ax, x, y, t, highlight=False):
    ax.add_patch(Rectangle((0, 0), 0.5, 1, fc=style.BLUE, alpha=0.08, lw=0))
    ax.axvline(0.5, color=style.MUTED, lw=1, ls=":")
    left = x < 0.5
    ax.scatter(x[left], y[left], s=60, color=style.BLUE, edgecolor="white", lw=0.8, zorder=3)
    ax.scatter(x[~left], y[~left], s=60, color=style.ORANGE, edgecolor="white", lw=0.8, zorder=3)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.set_aspect("equal")
    ax.set_xticks([])
    ax.set_yticks([])
    for sp in ax.spines.values():
        sp.set_visible(True)
        sp.set_linewidth(3 if highlight else 1.5)
        sp.set_color(style.GREEN if highlight else style.INK)
    ax.set_title(f"t = {t:g}" + ("   all back!" if highlight else ""),
                 color=style.GREEN if highlight else style.INK, fontsize=13)


def fig_snapshots(rng):
    n = 10
    x0, y0, vx, vy = make_gas(n, rng)
    k_ret = int(visits_to_left_half(x0, vx, 200_000)[0])
    print(f"[snapshots] N={n}: first return to the left half at step {k_ret} (Kac average 2^{n} = {2**n})")
    times = [0, 0.5, 2, 30, k_ret]
    fig, axes = plt.subplots(1, len(times), figsize=(3.0 * len(times), 3.3))
    for ax, t in zip(axes, times):
        x, y = positions(x0, y0, vx, vy, t)
        draw_box(ax, x[0], y[0], t, highlight=(t == k_ret))
    fig.suptitle(f"{n} particles start in the left half. They spread out. "
                 f"After {k_ret} looks, they are all back on the left.",
                 fontsize=14, weight="bold", y=1.03)
    style.save(fig, "05_gas_snapshots.png")
    return (x0, y0, vx, vy), k_ret


def fig_left_fraction(rng):
    t = np.arange(0, 6001) * DT
    fig, axes = plt.subplots(2, 1, figsize=(11, 5.2), sharex=True)
    for ax, n, col in zip(axes, (10, 40), (style.BLUE, style.PURPLE)):
        x0, _, vx, _ = make_gas(n, rng)
        x = bounce(x0[None, :] + vx[None, :] * t[:, None])
        frac = np.mean(x < 0.5, axis=1)
        ax.plot(t, frac, lw=0.5, color=col)
        back = np.nonzero(frac[1:] == 1.0)[0] + 1
        ax.plot(t[back], frac[back], "v", color=style.GREEN, ms=8)
        ax.axhline(0.5, color=style.MUTED, lw=1, ls=":")
```

```
        ax.set_ylim(0, 1.12)
        ax.set_ylabel("fraction on left")
        msg = (f"N = {n}: {len(back)} returns (green) in {len(t)-1} looks"
               if len(back) else f"N = {n}: no return in {len(t)-1} looks "
                                 f"(expected wait about $2^{{{n}}} \\approx
     10^{{{int(round(n*np.log10(2)))}}}$)")
        ax.set_title(msg, loc="left", fontsize=13)
    axes[-1].set_xlabel("time (number of looks)")
    fig.tight_layout()
    style.save(fig, "05_left_fraction.png")


def fig_kac(rng):
    Ns = np.arange(1, 15)
    rows = []
    for n in Ns:
        x0, _, vx, _ = make_gas(n, rng)
        n_steps = int(min(4000 * 2.0**n, 6e7))
        hits = visits_to_left_half(x0, vx, n_steps)
        gaps = np.diff(hits)
        mean = gaps.mean()
        err = gaps.std(ddof=1) / np.sqrt(len(gaps))
        rows.append((int(n), len(gaps), mean, err, 2.0**n))
        print(f"N={n:2d}  returns={len(gaps):7d}  mean gap={mean:10.1f} +/- {err:7.1f}   Kac 2^N={2**n}")

    with open(style.FIG_DIR / "05_results.csv", "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["N", "returns_counted", "mean_return_steps", "std_error", "kac_prediction"])
        w.writerows(rows)

    r = np.array(rows)
    fig, ax = plt.subplots(figsize=(7.6, 5))
    nn = np.linspace(1, 14, 100)
    ax.semilogy(nn, 2.0**nn, color=style.ORANGE, lw=2.5, label="Kac's prediction $2^N$", zorder=1)
    ax.errorbar(r[:, 0], r[:, 2], yerr=r[:, 3], fmt="o", ms=8, color=style.BLUE,
                capsize=3, label="our simulation", zorder=2)
    ax.set_xlabel("number of particles $N$")
    ax.set_ylabel("mean wait between returns (looks)")
    ax.set_title("Waiting time doubles with every particle")
    ax.legend(loc="upper left")
    ax.set_xticks(Ns)
    style.save(fig, "05_kac_scaling.png")

    # The punchline numbers
    ms_per_look = 1e-3
    year = 3.156e7
    for n in (100, 1000):
        yrs = 2.0**n * ms_per_look / year
        print(f"N={n}: 2^N looks at one per millisecond = {yrs:.2e} years "
              f"= {yrs/1.38e10:.1e} x age of universe")
    return rows


def gif_gas(gas, k_ret):
    x0, y0, vx, vy = gas
    early = np.linspace(0, 6, 60)
    late = np.linspace(k_ret - 6, k_ret, 60)
    times = list(early) + [None] * 6 + list(late) + [k_ret] * 12
    fig, ax = plt.subplots(figsize=(4.2, 4.5))

    def update(i):
        ax.clear()
        t = times[i]
        if t is None:
            ax.text(0.5, 0.5, f"... fast forward\nto t = {k_ret - 6:g} ...", ha="center",
                    va="center", fontsize=15, transform=ax.transAxes)
            ax.set_xticks([]), ax.set_yticks([])
            return
        x, y = positions(x0, y0, vx, vy, t)
        draw_box(ax, x[0], y[0], round(t, 1), highlight=bool(np.all(x[0] < 0.5) and t > 1))

    anim = FuncAnimation(fig, update, frames=len(times))
    anim.save(style.FIG_DIR / "05_gas.gif", writer=PillowWriter(fps=12), dpi=80)
    plt.close(fig)
    print("saved figures/05_gas.gif")
```

```

def main():
    style.apply()
    rng = np.random.default_rng(RNG_SEED)
    gas, k_ret = fig_snapshots(rng)
    fig_left_fraction(rng)
    fig_kac(rng)
    gif_gas(gas, k_ret)


if __name__ == "__main__":
    main()
```

B.2 06_gas_torus.py: the gas drawn on a torus

```
"""
The gas in a box, drawn as a point moving on a torus.

Step 1: one particle bouncing between the walls x = 0 and x = 1 is the same as
        a point running round a circle of length 2. Going once round the circle
        is: left wall -> right wall -> back to the left wall.
        We use the angle  phi = (theta + 1/2) mod 2,  where  theta = x0 + v t
        is the unfolded position. Then
            phi in [0, 1)  <=>  particle is in the LEFT half of the box
            phi in [1, 2)  <=>  particle is in the RIGHT half
Step 2: two particles = two angles (phi1, phi2) = a point on a torus.
        Cut open, the torus is the square [0, 2) x [0, 2) with opposite edges glued.
        The gas just moves in a straight line with slope v2 / v1, wrapping round.
Step 3: "both particles on the left" is the square [0, 1) x [0, 1):
        a quarter of the torus. Kac's lemma: mean wait = 1 / (1/4) = 4 looks.
        N particles: N-torus, target is 2^-N of it, mean wait 2^N looks.

Outputs
  figures/06_torus_idea.png         circle -> square torus -> doughnut, for N = 2
  figures/06_torus_dimensions.png   square (1/4) -> cube (1/8) -> 2^-N
  figures/06_torus_gas.gif          the box and the torus moving together
"""
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter
from matplotlib.patches import Rectangle, FancyArrowPatch, Wedge

import style

V = np.array([0.6180339887, 0.4142135624])     # velocities (units: box widths per look)
X0 = np.array([0.15, 0.35])                    # both start in the left half
N_LOOKS_DEMO = 14


# ---------------------------------------------------------------- physics
def phi(t):
    """Torus angles in [0, 2) at time(s) t, shape (len(t), 2)."""
    t = np.atleast_1d(t)[:, None]
    return (X0 + V * t + 0.5) % 2.0


def x_of_phi(ph):
    th = (ph - 0.5) % 2.0
    return np.where(th < 1.0, th, 2.0 - th)


def both_left(ph):
    return np.all(ph < 1.0, axis=-1)


def mean_wait(n_looks=200_000):
    ph = phi(np.arange(1, n_looks + 1))
    hits = np.nonzero(both_left(ph))[0]
    gaps = np.diff(hits)
    return gaps.mean(), len(gaps)

```

```
57
58  # ---------------------------------------------------------------- drawing helpers
59  def path_segments(t0, t1, n=4000):
60      """Straight-line path on the square, split where it wraps round an edge."""
61      ph = phi(np.linspace(t0, t1, n))
62      jump = np.any(np.abs(np.diff(ph, axis=0)) > 1.0, axis=1)
63      ph = ph.astype(float)
64      out = ph.copy()
65      out[1:][jump] = np.nan
66      return out
67
68
69  def draw_square(ax, n_looks, show_numbers=True, fs=12):
70      ax.add_patch(Rectangle((0, 0), 2, 2, fc="white", ec=style.INK, lw=2))
71      ax.add_patch(Rectangle((0, 0), 1, 1, fc=style.BLUE, alpha=0.16, ec="none"))
72      ax.text(0.5, 0.5, "both on\nthe left", ha="center", va="center", color=style.BLUE,
73              fontsize=fs + 1, weight="bold")
74      seg = path_segments(0, n_looks)
75      ax.plot(seg[:, 0], seg[:, 1], color=style.MUTED, lw=1.0, alpha=0.8)
76      ph = phi(np.arange(n_looks + 1))
77      inside = both_left(ph)
78      for k, (a, b) in enumerate(ph):
79          col = style.ORANGE if (inside[k] and k > 0) else (style.GREEN if k == 0 else style.INK)
80          ax.plot(a, b, "o", ms=9 if k == 0 or inside[k] else 6, color=col, zorder=5, mec="white")
81          if show_numbers:
82              ax.text(a + 0.05, b + 0.05, str(k), fontsize=fs - 2, color=col, zorder=6)
83      # glued-edge arrows: single arrows on left/right, double on top/bottom
84      for y in (0.0, 2.0):
85          ax.annotate("", xy=(1.15, y), xytext=(0.85, y),
86                      arrowprops=dict(arrowstyle="-|>", color=style.PURPLE, lw=2.2))
87      for x in (0.0, 2.0):
88          ax.annotate("", xy=(x, 1.15), xytext=(x, 0.85),
89                      arrowprops=dict(arrowstyle="-|>", color=style.RED, lw=2.2))
90      ax.set_xlim(-0.08, 2.08)
91      ax.set_ylim(-0.08, 2.08)
92      ax.set_aspect("equal")
93      ax.set_xticks([0, 1, 2], ["0", "1", "2"])
94      ax.set_yticks([0, 1, 2], ["0", "1", "2"])
95      ax.set_xlabel("particle 1 angle $\\varphi_1$")
96      ax.set_ylabel("particle 2 angle $\\varphi_2$")
97      return inside
98
99
100 def draw_circle(ax):
101     R = 1.0
102     ax.add_patch(Wedge((0, 0), R, 90, 270, width=0.16, fc=style.BLUE, ec="none"))   # phi in [0,1): left
103     ax.add_patch(Wedge((0, 0), R, -90, 90, width=0.16, fc=style.ORANGE, ec="none"))  # phi in [1,2): right
104     # angle on the circle: phi = 0 at the top, increasing anticlockwise; half a turn = 1
105     def pt(p, r=R):
106         a = np.pi / 2 + np.pi * p
107         return r * np.cos(a), r * np.sin(a)
108     for p, lab, off in [(0.5, "left wall\n$x = 0$", 1.32), (1.5, "right wall\n$x = 1$", 1.32),
109                         (0.0, "middle", 1.2), (1.0, "middle", 1.2)]:
110         x, y = pt(p, R - 0.08)
111         ax.plot(x, y, "o", color=style.INK, ms=6, zorder=5)
112         lx, ly = pt(p, off)
113         ax.text(lx, ly, lab, ha="center", va="center", fontsize=11)
114     ax.add_patch(FancyArrowPatch(pt(0.12, 0.72), pt(0.38, 0.72), connectionstyle="arc3,rad=0.25",
115                                  arrowstyle="-|>", mutation_scale=16, color=style.INK, lw=1.5))
116     ax.text(-0.45, 0, "left\nhalf", color=style.BLUE, ha="center", va="center", fontsize=13, weight="bold")
117     ax.text(0.45, 0, "right\nhalf", color=style.ORANGE, ha="center", va="center", fontsize=13, weight="bold")
118     ax.text(0, -1.55, "middle = the line $x = 1/2$", ha="center", fontsize=10.5, color=style.MUTED)
119     ax.set_xlim(-1.75, 1.75)
120     ax.set_ylim(-1.7, 1.6)
121     ax.set_aspect("equal")
122     ax.axis("off")
123
124
125 def draw_doughnut(ax, n_looks):
126     R, r = 3.0, 1.15
127     elev, azim = 40, -60
128     u, v = np.meshgrid(np.linspace(0, 2 * np.pi, 120), np.linspace(0, 2 * np.pi, 60))
129     X = (R + r * np.cos(v)) * np.cos(u)
130     Y = (R + r * np.cos(v)) * np.sin(u)
131     Z = r * np.sin(v)
```

```
    target = (u < np.pi) & (v < np.pi)                  # phi1 in [0,1), phi2 in [0,1)
    cols = np.where(target[..., None], np.array([0.55, 0.68, 0.96, 1.0]), np.array([0.90, 0.91, 0.93, 1.0]))
    ax.plot_surface(X, Y, Z, facecolors=cols, linewidth=0, shade=True, antialiased=True)

    e, az = np.radians(elev), np.radians(azim)
    view = np.array([np.cos(e) * np.cos(az), np.cos(e) * np.sin(az), np.sin(e)])

    def place(ph, lift):
        uu, vv = np.pi * ph[:, 0], np.pi * ph[:, 1]
        rr = r + lift
        P = np.stack([(R + rr * np.cos(vv)) * np.cos(uu), (R + rr * np.cos(vv)) * np.sin(uu), rr *
     np.sin(vv)])
        normal = np.stack([np.cos(vv) * np.cos(uu), np.cos(vv) * np.sin(uu), np.sin(vv)])
        return P, view @ normal > 0

    P, front = place(phi(np.linspace(0, n_looks, 4000)), 0.03)
    ax.plot(*np.where(front, P, np.nan), color=style.INK, lw=1.2)      # only the side facing us
    pk = phi(np.arange(n_looks + 1))
    ins = both_left(pk)
    Pk, fk = place(pk, 0.08)
    for k in range(len(pk)):
        if not fk[k]:
            continue
        col = style.GREEN if k == 0 else (style.ORANGE if ins[k] else style.INK)
        ax.scatter(*Pk[:, k], color=col, s=90 if (k == 0 or ins[k]) else 25, depthshade=False,
                   edgecolors="white", linewidths=0.8)
    ax.set_box_aspect((1, 1, 0.4), zoom=1.35)
    ax.view_init(elev=elev, azim=azim)
    ax.computed_zorder = False
    ax.axis("off")


# ---------------------------------------------------------------- figures
def fig_idea():
    fig = plt.figure(figsize=(16.5, 5.6))
    ax1 = fig.add_subplot(1, 3, 1)
    draw_circle(ax1)
    ax1.set_title("1. One particle: a bounce is a circle", pad=12)
    ax2 = fig.add_subplot(1, 3, 2)
    inside = draw_square(ax2, N_LOOKS_DEMO)
    ax2.set_title("2. Two particles: a point on a torus\n(square with opposite edges glued)", pad=10)
    ax3 = fig.add_subplot(1, 3, 3, projection="3d")
    draw_doughnut(ax3, N_LOOKS_DEMO)
    ax3.set_title("3. Glue the edges: a doughnut\n(blue = both on the left; front side shown)", pad=0)
    first = int(np.nonzero(inside[1:])[0][0]) + 1
    fig.text(0.5, -0.06,
             f"Green dot = start (look 0). Orange = both particles back on the left (first at look {first}).
     "
             "Blue patch = a quarter of the torus.", ha="center", fontsize=13, color=style.INK)
    fig.tight_layout()
    style.save(fig, "06_torus_idea.png")
    return first


def fig_dimensions():
    fig = plt.figure(figsize=(13, 4.8))
    ax = fig.add_subplot(1, 3, 1)
    ax.add_patch(Rectangle((0, 0), 2, 2, fc="white", ec=style.INK, lw=2))
    ax.add_patch(Rectangle((0, 0), 1, 1, fc=style.BLUE, alpha=0.35, ec=style.BLUE, lw=1.5))
    ax.set_xlim(-0.1, 2.1)
    ax.set_ylim(-0.1, 2.1)
    ax.set_aspect("equal")
    ax.axis("off")
    ax.set_title("$N = 2$: square torus\ntarget = $1/4$, wait = 4 looks")

    ax = fig.add_subplot(1, 3, 2, projection="3d")
    def cube(ax, o, s, face, edge, alpha):
        o = np.array(o, float)
        for a in range(3):
            for side in (0, 1):
                idx = [i for i in range(3) if i != a]
                g = np.linspace(0, s, 2)
                A, B = np.meshgrid(g, g)
                P = [None] * 3
                P[a] = np.full_like(A, o[a] + side * s)
```

```
                P[idx[0]] = o[idx[0]] + A
                P[idx[1]] = o[idx[1]] + B
                ax.plot_surface(*P, color=face, alpha=alpha, edgecolor=edge, linewidth=1.2, shade=False)
    cube(ax, (0, 0, 0), 2, "white", style.INK, 0.05)
    cube(ax, (0, 0, 0), 1, style.BLUE, style.BLUE, 0.45)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=22, azim=-58)
    ax.axis("off")
    ax.set_title("$N = 3$: cube torus\ntarget = $1/8$, wait = 8 looks", pad=0)

    ax = fig.add_subplot(1, 3, 3)
    ax.axis("off")
    ax.text(0.5, 0.62, "$N$ particles", ha="center", fontsize=20, weight="bold")
    ax.text(0.5, 0.44, "$N$-torus,  target $= 2^{-N}$", ha="center", fontsize=19)
    ax.text(0.5, 0.26, "mean wait $= 2^{N}$ looks", ha="center", fontsize=19, color=style.ORANGE)
    fig.tight_layout()
    style.save(fig, "06_torus_dimensions.png")


def gif_gas():
    fpl = 6                                     # frames per look
    n_looks = 30
    times = np.arange(n_looks * fpl + 1) / fpl
    ph_all = phi(times)
    xs = x_of_phi(ph_all)
    looks = phi(np.arange(n_looks + 1))
    ins = both_left(looks)

    fig, (axb, axt) = plt.subplots(1, 2, figsize=(9.6, 4.9), gridspec_kw={"width_ratios": [1, 1.05]})
    seg = path_segments(0, n_looks, 6000)

    def update(i):
        t = times[i]
        k = int(np.floor(t + 1e-9))
        for ax in (axb, axt):
            ax.clear()
        # the box
        axb.add_patch(Rectangle((0, 0), 0.5, 1, fc=style.BLUE, alpha=0.12, ec="none"))
        axb.add_patch(Rectangle((0, 0), 1, 1, fc="none", ec=style.INK, lw=2))
        axb.axvline(0.5, color=style.MUTED, ls=":", lw=1)
        axb.plot(xs[i, 0], 0.65, "o", ms=16, color=style.PURPLE, mec="white", zorder=5)
        axb.plot(xs[i, 1], 0.35, "o", ms=16, color=style.GREEN, mec="white", zorder=5)
        axb.text(0.02, 0.92, "particle 1", color=style.PURPLE, fontsize=11, transform=axb.transAxes)
        axb.text(0.02, 0.05, "particle 2", color=style.GREEN, fontsize=11, transform=axb.transAxes)
        axb.set_xlim(-0.02, 1.02)
        axb.set_ylim(-0.02, 1.02)
        axb.set_aspect("equal")
        axb.set_xticks([])
        axb.set_yticks([])
        for sp in axb.spines.values():
            sp.set_visible(False)
        n_ret = int(ins[1:k + 1].sum())
        axb.set_title(f"the box   (look {k}, returns so far: {n_ret})", fontsize=12)
        # the torus
        axt.add_patch(Rectangle((0, 0), 2, 2, fc="white", ec=style.INK, lw=2))
        axt.add_patch(Rectangle((0, 0), 1, 1, fc=style.BLUE, alpha=0.16, ec="none"))
        upto = int(t / n_looks * (len(seg) - 1))
        axt.plot(seg[:upto + 1, 0], seg[:upto + 1, 1], color=style.MUTED, lw=1.0)
        for j in range(k + 1):
            col = style.GREEN if j == 0 else (style.ORANGE if ins[j] else style.INK)
            axt.plot(*looks[j], "o", ms=7 if (j == 0 or ins[j]) else 4, color=col, zorder=5)
        axt.plot(*ph_all[i], "o", ms=12, color=style.RED, mec="white", zorder=6)
        axt.set_xlim(-0.05, 2.05)
        axt.set_ylim(-0.05, 2.05)
        axt.set_aspect("equal")
        axt.set_xticks([0, 1, 2])
        axt.set_yticks([0, 1, 2])
        axt.set_xlabel("$\\varphi_1$")
        axt.set_ylabel("$\\varphi_2$")
        axt.set_title("the same gas: one point on a torus", fontsize=12)

    frames = list(range(len(times))) + [len(times) - 1] * 12
    FuncAnimation(fig, lambda f: update(frames[f]), frames=len(frames)).save(
        style.FIG_DIR / "06_torus_gas.gif", writer=PillowWriter(fps=12), dpi=70)
    plt.close(fig)
```

```
    print("saved figures/06_torus_gas.gif")


def main():
    style.apply()
    m, n = mean_wait()
    print(f"N = 2: mean wait between returns = {m:.3f} looks over {n} returns (Kac: 4)")
    first = fig_idea()
    print(f"first return to 'both on the left' at look {first}")
    fig_dimensions()
    gif_gas()


if __name__ == "__main__":
    main()
```

B.3 07_torus_figures.py: filling and dimension pictures

```
"""
Two more torus pictures for the gas in a box (N = 2 and N = 3 use the same looks as 06_gas_torus.py).

  figures/06_torus_filling.png   after 20, 100 and 400 looks the dots cover the square evenly,
                                 so about 1 look in 4 lands in the "both on the left" quarter
  figures/06_torus_steps.png     circle -> square -> cube: each new particle adds a direction
                                 and halves the target (1/2, 1/4, 1/8)
"""
import importlib
import sys
from pathlib import Path

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle, FancyArrowPatch

sys.path.insert(0, str(Path(__file__).resolve().parent))
import style

m = importlib.import_module("06_gas_torus")

# a third velocity and start point, used only for the N = 3 panel
V = np.array([0.6180339887, 0.4142135624, 0.7320508076])
X0 = np.array([0.15, 0.35, 0.25])
L = 120


def phi(n, d):
    k = np.arange(n)[:, None]
    return (X0[:d] + V[:d] * k + 0.5) % 2.0


def fig_filling():
    fig, axes = plt.subplots(1, 3, figsize=(15, 5.4))
    for ax, n in zip(axes, (20, 100, 400)):
        ph = m.phi(np.arange(n)); ins = m.both_left(ph)
        ax.add_patch(Rectangle((0, 0), 2, 2, fc="white", ec=style.INK, lw=2))
        ax.add_patch(Rectangle((0, 0), 1, 1, fc=style.BLUE, alpha=0.15, ec=style.BLUE, lw=1.5))
        for x in (1,): ax.axvline(x, color=style.MUTED, lw=0.8, ls=":"); ax.axhline(x, color=style.MUTED,
     lw=0.8, ls=":")
        ax.plot(ph[~ins, 0], ph[~ins, 1], "o", ms=5, color=style.INK, alpha=0.7)
        ax.plot(ph[ins, 0], ph[ins, 1], "o", ms=6, color=style.ORANGE)
        k = int(ins.sum())
        ax.set_title(f"{n} looks: {k} land in the blue quarter\n{n} / {k} = {n/k:.1f} looks per visit",
     fontsize=14)
        ax.set_xlim(-0.05, 2.05); ax.set_ylim(-0.05, 2.05); ax.set_aspect("equal")
        ax.set_xticks([0, 1, 2]); ax.set_yticks([0, 1, 2])
        print(f'{n} looks: {k} in the target, {n / k:.2f} looks per visit')
    fig.suptitle("The dot spreads evenly over the torus, so it lands in a quarter of the area a quarter of
     the time",
                 fontsize=15, weight="bold", y=1.02)
    fig.tight_layout()
    style.save(fig, "06_torus_filling.png")

```

```
52
53  def fig_steps():
54      fig = plt.figure(figsize=(17, 6.2))
55      gs = fig.add_gridspec(1, 5, width_ratios=[1, 0.18, 1, 0.18, 1.15])
56
57      # ---- 1D: a line segment [0,2) with its ends glued = circle
58      ax = fig.add_subplot(gs[0])
59      p = phi(L, 1)[:, 0]
60      ins = p < 1
61      ax.plot([0, 2], [0, 0], color=style.INK, lw=2.5)
62      ax.plot([0, 1], [0, 0], color=style.BLUE, lw=9, alpha=0.35, solid_capstyle="butt")
63      jit = np.linspace(-0.25, 0.25, L)        # spread dots vertically only so they don't overlap
64      ax.scatter(p[~ins], 0.0 * p[~ins] + jit[~ins] * 0 + 0.12 + 0.0, s=0)  # placeholder
65      ax.plot(p[~ins], np.zeros(ins.size)[~ins] + 0.18, "|", ms=14, color=style.INK, alpha=0.6)
66      ax.plot(p[ins], np.zeros(ins.size)[ins] + 0.18, "|", ms=14, color=style.ORANGE)
67      for x in (0, 2):
68          ax.plot(x, 0, "o", ms=10, color=style.PURPLE, zorder=5)
69      ax.annotate("", xy=(2, -0.5), xytext=(0, -0.5),
70                  arrowprops=dict(arrowstyle="<->", color=style.PURPLE, lw=1.5,
     connectionstyle="arc3,rad=0.35"))
71      ax.text(1, -0.82, "glue the two ends: a circle", ha="center", color=style.PURPLE, fontsize=12)
72      ax.text(0.5, 0.45, "left half", ha="center", color=style.BLUE, fontsize=13, weight="bold")
73      ax.set_xlim(-0.15, 2.15); ax.set_ylim(-1.0, 0.8)
74      ax.set_xticks([]); ax.set_yticks([])
75      for sp in ax.spines.values(): sp.set_visible(False)
76      for x in (0, 1, 2): ax.text(x, -0.14, str(x), ha="center", va="top", fontsize=13)
77      ax.text(1, -0.3, "$\\varphi_1$", ha="center", va="top", fontsize=14)
78      k1 = int(ins.sum())
79      ax.set_title(f"1 particle: a circle\ntarget = 1/2,  {k1} of {L} looks inside", fontsize=14)
80
81      def arrow(ax, text):
82          ax.axis("off")
83          ax.add_patch(FancyArrowPatch((0.05, 0.5), (0.95, 0.5), transform=ax.transAxes, arrowstyle="-|>",
84                                       mutation_scale=28, color=style.INK, lw=2.5))
85          ax.text(0.5, 0.62, text, ha="center", va="bottom", transform=ax.transAxes, fontsize=12)
86
87      arrow(fig.add_subplot(gs[1]), "add a\nparticle\n= add a\ncircle")
88
89      # ---- 2D
90      ax = fig.add_subplot(gs[2])
91      p = phi(L, 2); ins = np.all(p < 1, axis=1)
92      ax.add_patch(Rectangle((0, 0), 2, 2, fc="white", ec=style.INK, lw=2))
93      ax.add_patch(Rectangle((0, 0), 1, 1, fc=style.BLUE, alpha=0.18, ec=style.BLUE, lw=1.5))
94      ax.plot(p[~ins, 0], p[~ins, 1], "o", ms=4, color=style.INK, alpha=0.6)
95      ax.plot(p[ins, 0], p[ins, 1], "o", ms=5.5, color=style.ORANGE)
96      ax.plot([0, 2], [-0.12, -0.12], color=style.INK, lw=0)
97      ax.set_xlim(-0.05, 2.05); ax.set_ylim(-0.05, 2.05); ax.set_aspect("equal")
98      ax.set_xticks([0, 1, 2]); ax.set_yticks([0, 1, 2])
99      ax.set_xlabel("$\\varphi_1$"); ax.set_ylabel("$\\varphi_2$")
100     ax.text(0.5, 0.5, "1/2 × 1/2", ha="center", va="center", color=style.BLUE, fontsize=13, weight="bold",
101             bbox=dict(fc="white", ec="none", alpha=0.85, pad=2), zorder=6)
102     k2 = int(ins.sum())
103     ax.set_title(f"2 particles: a square (doughnut)\ntarget = 1/4,  {k2} of {L} looks inside", fontsize=14)
104
105     arrow(fig.add_subplot(gs[3]), "add a\nparticle\n= add a\ncircle")
106
107     # ---- 3D
108     ax = fig.add_subplot(gs[4], projection="3d")
109     p = phi(L, 3); ins = np.all(p < 1, axis=1)
110     def box(o, s, color, alpha, lw):
111         o = np.array(o, float)
112         r = [0, s]
113         for a in range(3):
114             for b in range(3):
115                 if b <= a: continue
116                 c = 3 - a - b
117                 for u in r:
118                     for w in r:
119                         P = np.zeros((2, 3)); P[:, a] = u; P[:, b] = w; P[:, c] = r
120                         ax.plot(*(P + o).T, color=color, alpha=alpha, lw=lw)
121     box((0, 0, 0), 2, style.INK, 0.6, 1.5)
122     # shaded target cube faces
123     g = np.linspace(0, 1, 2); A, B = np.meshgrid(g, g)
124     for a in range(3):
125         for side in (0, 1):
```

```
            P = [None] * 3; idx = [i for i in range(3) if i != a]
            P[a] = np.full_like(A, side * 1.0); P[idx[0]] = A; P[idx[1]] = B
            ax.plot_surface(*P, color=style.BLUE, alpha=0.18, linewidth=0, shade=False)
    ax.scatter(*p[~ins].T, s=10, color=style.INK, alpha=0.5, depthshade=False)
    ax.scatter(*p[ins].T, s=30, color=style.ORANGE, depthshade=False)
    ax.set_xticks([0, 1, 2]); ax.set_yticks([0, 1, 2]); ax.set_zticks([0, 1, 2])
    ax.set_xlabel("$\\varphi_1$"); ax.set_ylabel("$\\varphi_2$"); ax.set_zlabel("$\\varphi_3$")
    ax.view_init(elev=20, azim=-60); ax.set_box_aspect((1, 1, 1), zoom=0.92)
    k3 = int(ins.sum())
    ax.set_title(f"3 particles: a cube\ntarget = 1/8,  {k3} of {L} looks inside", fontsize=14, pad=0)

    fig.suptitle("Each new particle adds one more circle, so one more direction. The target is the left half
     "
                 "along every direction, so it halves each time.", fontsize=14.5, weight="bold", y=1.02)
    style.save(fig, "06_torus_steps.png")
    print(f'looks inside the target out of {L}: circle {k1}, square {k2}, cube {k3}')


def main():
    style.apply()
    fig_filling()
    fig_steps()


if __name__ == "__main__":
    main()
```

B.4 `style.py`: shared plot style

```
"""Shared plot style so every figure in the deck looks like it came from one team."""
from pathlib import Path

import matplotlib as mpl
import matplotlib.pyplot as plt

FIG_DIR = Path(__file__).resolve().parent.parent / "figures"
FIG_DIR.mkdir(exist_ok=True)

# A small, high-contrast palette that survives a washed-out projector.
INK = "#1b1f2a"
MUTED = "#6b7280"
BLUE = "#2563eb"
ORANGE = "#ea580c"
GREEN = "#059669"
PURPLE = "#7c3aed"
RED = "#dc2626"
PALETTE = [BLUE, ORANGE, GREEN, PURPLE, RED]


def apply():
    mpl.rcParams.update({
        "figure.dpi": 110,
        "savefig.dpi": 200,
        "savefig.bbox": "tight",
        "font.family": "DejaVu Sans",
        "font.size": 13,
        "axes.titlesize": 15,
        "axes.titleweight": "bold",
        "axes.labelsize": 13,
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.prop_cycle": mpl.cycler(color=PALETTE),
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "legend.frameon": False,
        "mathtext.fontset": "cm",
    })


```

```python
def save(fig, name):
    path = FIG_DIR / name
    fig.savefig(path)
    plt.close(fig)
    print(f"saved {path.relative_to(FIG_DIR.parent)}")
    return path
```